1. 메시지, 토픽, 서비스, 액션, 인터페이스 용어 설명

1) 메시지(Message)

메시지는 ROS2에서 노드와 노드 사이에 주고받는 데이터이다.

예를 들어 거북이를 움직일 때 이동 속도와 회전 속도 등의 정보를 메시지에 담아서 전달한다.

쉽게 말하면 메시지는 노드끼리 주고받는 '내용물'이라고 할 수 있다.

2) 토픽(Topic)

토픽은 노드와 노드 사이에서 메시지를 계속 주고받을 수 있도록 만들어진 통신 통로이다.

메시지를 보내는 것을 Publish(발행), 메시지를 받는 것을 Subscribe(구독)라고 한다.

예를 들어 `/turtle1/cmd_vel` 토픽은 거북이에게 이동 명령을 전달하는 데 사용된다.

쉽게 말하면 토픽은 '메시지가 지나가는 통로'라고 할 수 있다.

3) 서비스(Service)

서비스는 특정 작업을 요청하고 그 결과를 한 번 응답받는 통신 방식이다.

클라이언트가 요청(Request)을 보내면 서버가 작업을 수행한 후 응답(Response)을 보내준다.

예를 들어 turtlesim에서 새로운 거북이를 생성하는 `/spawn` 서비스가 있다.

쉽게 말하면 서비스는 '질문을 보내고 답을 받는 방식'이라고 할 수 있다.

4) 액션(Action)

액션은 시간이 오래 걸리는 작업을 수행할 때 사용하는 통신 방식이다.

서비스와 달리 작업이 진행되는 동안 현재 진행 상황을 확인할 수 있으며, 필요한 경우 작업을 중간에 취소할 수도 있다.

쉽게 말하면 액션은 '시간이 오래 걸리는 작업을 요청하고 진행 상황까지 확인하는 방식'이다.

5) 인터페이스(Interface)

인터페이스는 ROS2에서 노드끼리 데이터를 주고받을 때 사용하는 데이터의 형식과 구조를 정의한 것이다.

메시지, 서비스, 액션이 어떤 데이터를 주고받을 것인지 정해놓은 규칙이라고 할 수 있다.

예를 들어 `geometry_msgs/msg/Twist` 인터페이스에는 직선 이동 속도와 회전 속도에 대한 데이터 구조가 정의되어 있다.

2. turtlesim과 teleop_turtle 노드 실행

ROS2의 turtlesim과 teleop_turtle 노드를 각각 실행하였다.

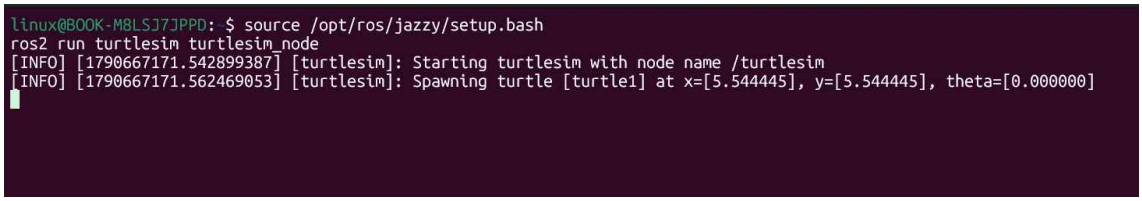
turtlesim 노드는 다음 명령어를 이용하여 실행하였다.

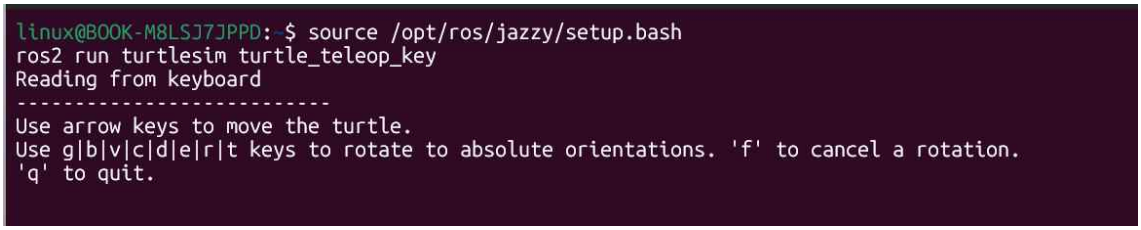
`ros2 run turtlesim turtlesim_node`

teleop_turtle 노드는 다음 명령어를 이용하여 실행하였다.

`ros2 run turtlesim turtle_teleop_key`

teleop_turtle이 실행된 터미널에서 방향키를 입력하였으며, 방향키 입력에 따라 turtlesim의 거북이가 움직이는 것을 확인하였다.





3. 현재 실행 중인 토픽과 메시지 인터페이스 확인

현재 실행 중인 토픽과 각 토픽에서 사용하는 메시지 인터페이스를 확인하기 위해 다음 명령어를 사용하였다.

`ros2 topic list -t`

실행 결과 turtlesim에서 다음과 같은 주요 토픽과 메시지 인터페이스를 확인할 수 있었다.

`/turtle1/cmd_vel` → `geometry_msgs/msg/Twist`

`/turtle1/color_sensor` → `turtlesim/msg/Color`

`/turtle1/pose` → `turtlesim/msg/Pose`

`/turtle1/cmd_vel`은 거북이의 이동 및 회전 명령을 전달하는 토픽이다.

`/turtle1/color_sensor`는 거북이가 위치한 곳의 배경 색상 정보를 전달하는 토픽이다.

`/turtle1/pose`는 거북이의 현재 위치, 방향, 이동 속도 등의 정보를 전달하는 토픽이다.

```
linux@BOOK-M8LSJ7JPPD:~$ source /opt/ros/jazzy/setup.bash

ros2 topic list

ros2 topic list -t

ros2 topic echo /turtle1/pose
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
```

[여기에 ros2 topic list -t 실행 결과 캡처 삽입]

4. 토픽 메시지 확인

거북이의 현재 위치와 움직임에 대한 메시지를 확인하기 위해 다음 명령어를 사용하였다.

`ros2 topic echo /turtle1/pose`

실행 결과 x, y, theta, linear_velocity, angular_velocity 등의 값이 출력되는 것을 확인하였다.

x와 y는 거북이의 현재 위치를 의미하며, theta는 거북이가 바라보는 방향을 의미한다.

linear_velocity는 직선 이동 속도이고, angular_velocity는 회전 속도를 의미한다.

```
linux@BOOK-M8LSJ7JPPD:~$ source /opt/ros/jazzy/setup.bash

ros2 topic list

ros2 topic list -t

ros2 topic echo /turtle1/pose
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 5.544444561004639
y: 5.544444561004639
```

5. 메시지 인터페이스 정의 확인

앞에서 확인한 메시지 인터페이스의 내부 데이터 구조를 확인하기 위해 `ros2 interface show` 명령어를 사용하였다.

1) geometry_msgs/msg/Twist

사용한 명령어:

`ros2 interface show geometry_msgs/msg/Twist`

Twist 인터페이스는 크게 `linear`과 `angular`로 구성되어 있다.

`linear`은 직선 방향의 이동 속도를 나타내고, `angular`은 회전 속도를 나타낸다.

따라서 turtlesim에서는 Twist 메시지를 이용하여 거북이의 이동 방향과 속도를 전달할 수 있다.

```
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
        float64 x
        float64 y
        float64 z
Vector3  angular
        float64 x
        float64 y
        float64 z
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show turtlesim/msg/Color
uint8 r
uint8 g
uint8 b
linux@BOOK-M8LSJ7JPPD:~$
```

2) turtlesim/msg/Pose

사용한 명령어:

`ros2 interface show turtlesim/msg/Pose`

Pose 인터페이스에는 다음과 같은 데이터가 정의되어 있다.

`float32 x`
`float32 y`
`float32 theta`
`float32 linear_velocity`
`float32 angular_velocity`

x와 y는 거북이의 위치를 나타내고, theta는 거북이의 방향을 나타낸다.

linear_velocity와 angular_velocity는 각각 직선 이동 속도와 회전 속도를 나타낸다.

```
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
        float64 x
        float64 y
        float64 z
Vector3  angular
        float64 x
        float64 y
        float64 z
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show turtlesim/msg/Color
uint8 r
uint8 g
uint8 b
linux@BOOK-M8LSJ7JPPD:~$
```

3) turtlesim/msg/Color

사용한 명령어:

`ros2 interface show turtlesim/msg/Color`

Color 인터페이스에는 다음과 같은 데이터가 정의되어 있다.

`uint8 r`
`uint8 g`
`uint8 b`

r은 빨간색(Red), g는 초록색(Green), b는 파란색(Blue)의 값을 의미한다.

이 값을 이용하여 거북이가 위치한 곳의 배경 색상 정보를 표현한다.

```
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
        float64 x
        float64 y
        float64 z
Vector3  angular
        float64 x
        float64 y
        float64 z
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
linux@BOOK-M8LSJ7JPPD:~$ ros2 interface show turtlesim/msg/Color
uint8 r
uint8 g
uint8 b
linux@BOOK-M8LSJ7JPPD:~$
```

[여기에 Color 인터페이스 출력 결과 캡처 삽입]

6. 실습 결과

이번 실습을 통해 ROS2에서 사용하는 메시지, 토픽, 서비스, 액션, 인터페이스의 차이를 이해할 수 있었다.

또한 turtlesim과 teleop_turtle 노드를 직접 실행하여 노드 사이에서 토픽을 통해 메시지가 전달되는 과정을 확인하였다.

`ros2 topic list -t` 명령어를 이용하여 토픽과 메시지 인터페이스를 확인하였고, `ros2 topic echo` 명령어를 이용하여 실제 전달되는 메시지의 값을 확인하였다.

마지막으로 `ros2 interface show` 명령어를 이용하여 각각의 메시지 인터페이스가 어떤 데이터 구조로 구성되어 있는지 확인하였다.